

CADERNO DO PROGRAMADOR — ÉVORA OVERSIGHT

2 de setembro de 2026 · acompanha o Manual Supremo v10.9.1 Confidencial — Eng. de Sistemas Luiz Gonzaga Filho

COMO USAR ESTE CADERNO

O Manual tem 151 páginas e explica **por quê**. Este caderno explica **o quê e em que ordem**.
Leia este primeiro. Consulte o Manual quando precisar entender a razão de uma trava — e há razão para todas.

A regra que governa o projeto inteiro: as travas ficam **no banco**, não no código. Se o programa pedir errado, o banco recusa. Não mova uma trava para a aplicação "para simplificar" — é exatamente o que este desenho existe para impedir.

PARTE 1 · O QUE JÁ EXISTE

Tudo abaixo foi **executado e testado** em PostgreSQL 16, em 02/09/2026.

Ordem	Arquivo	O que faz	Testes
1	banco_evora/evora_schema_mvp_v1.sql	11 tabelas base	—
2	banco_evora/evora_rls_mvp_v1.sql	13 políticas · RLS forçado	—
3	banco_evora/evora_teste_aceite_v1.sql	Prova de isolamento base	5
4	frontend_producao/sql/evora_token_hook.sql	Ponte login ↔ isolamento	3
5	banco_evora/evora_farus_f1.sql	Acervo compartilhado por município	—
6	banco_evora/evora_farus_teste_aceite.sql	Prova das travas do acervo	10
7	banco_evora/evora_alegis_a0.sql	Cofre de dado sensível	—

Ordem	Arquivo	O que faz	Testes
8	<code>banco_evora/evora_aceite_teste.sql</code>	Prova das travas do cofre	10

Rode nesta ordem. Cada um pressupõe o anterior.

PARTE 2 · A PEÇA MAIS CRÍTICA

O hook de token

As políticas de isolamento leem a identidade do cliente **de dentro do token**, em `app_metadata.tenant_id`. **O Supabase não coloca esse campo sozinho.**

Sem o hook ativado no painel, todo token nasce sem tenant e **o banco nega tudo**. O produto abre e não mostra nada.

Instalação: 1. Rodar `evora_token_hook.sql` 2. Painel → Authentication → Hooks → Customize Access Token (JWT) Claims → selecionar `public.evora_token_hook` → Enable 3. Logout e login de novo (o token antigo não tem os claims)

Verificação, com a sessão aberta:

```
select evora_tenant_atual() as tenant, evora_mundo_atual() as mundo;
```

Se vier `NULL`, o hook não está ativo ou o usuário não tem cadastro ativo.

Comportamento comprovado por teste: usuário ativo recebe `tenant_id`, `mundo` e `papel`. Usuário **inativo** ou **inexistente** recebe token **sem claims** — e o banco nega tudo. É falha fechada, e é o comportamento correto.

PARTE 3 · O FARUS

O problema que ele resolve

Hoje o coletor grava em `noticias.json`, arquivo local **sobrescrito a cada execução**. Cada briefing recomeça do zero, não há pesquisa histórica e não há economia de escala.

A exceção ao padrão de isolamento

Todas as 11 tabelas anteriores isolam por `tenant_id`. **O FARUS inverte:** o acervo público é compartilhado por **município**.

A fronteira é o campo `origem_aquisicao` :

Valor	Compartilhável	Significado
<code>coleta_programada</code>	sim	Rotina territorial — acervo público
<code>descoberta_radar</code>	sim	Radar autônomo (fase F3)
<code>interesse_tenant</code>	não	Disparado por interesse do cliente
<code>demanda_tenant</code>	não	Pesquisa sob demanda do cliente

Trava no banco: registro compartilhável **não pode** ter `tenant_origem` ; registro de tenant **precisa** ter. Provado pelo teste 7.

O que o motor precisa fazer

Trocar a escrita em arquivo por inserção em `farus_itens` , com:

```
insert into farus_itens (
  territorio_id, origem_aquisicao, especie, titulo, conteudo,
  url, publicado_em, fonte_id, estado, verificado_por, verificado_em, hash_conteudo
) values (
  :territorio, 'coleta_programada', 'ato_oficial', :titulo, :conteudo,
  :url, :data, :fonte, 'confirmado', 'motor', now(),
  farus_hash(:titulo, :conteudo)
)
on conflict (territorio_id, hash_conteudo) do nothing;
```

Três pontos que não podem ser alterados:

`farus_hash` é a **regra única de deduplicação**. Não calcule hash à parte — coletor e banco divergiriam. A função normaliza espaços, caixa e acentuação.

`on conflict do nothing` é o que impede a mesma matéria entrar duas vezes.

`estado` só pode ser `confirmado` ou `corroborado` **se houver verificador registrado**. O banco recusa o contrário.

Ciclo de vida da fonte

`descoberta` → `observada` → `configurada`

Fonte descoberta pelo radar entra como `observada` e **não sustenta afirmação de briefing**. Só vira `configurada` com `liberada_por` e `liberada_em` preenchidos — é a decisão D6 do Anexo 16 imposta pelo banco.

O que o briefing lê

Apenas itens em estado **confirmado** ou **corroborado**, de fontes **configuradas**. O RLS já filtra território e propriedade — **não acrescenta filtro de tenant na consulta**. O filtro é do banco.

PARTE 4 · O AEGIS

A regra que governa o módulo

Se o cofre precisar abrir na operação de rotina, não é cofre.

Nada no login, no briefing ou nas telas pode depender do conteúdo cofrado.

Como a aplicação usa

Para exibir — sempre, em qualquer tela:

```
select aegis_mascara(:usuario_id, 'cpf'); -- devolve '...982.247-...'
```

Não abre o cofre, não gera registro de acesso, qualquer papel pode chamar.

Para ler o valor real — raro, e sempre com motivo:

```
select aegis_ler(:usuario_id, 'cpf', 'emissao_contrato', 'contrato de adesão');
```

Grava o acesso **antes** de devolver. Apenas **autoridade** e **chefe_gabinete**.

O que NÃO funciona, por desenho:

```
select valor from aegis_cofre; -- permission denied
```

Não existe política de leitura no cofre. A ausência é a proteção, não esquecimento. Se alguém "corrigir" isso, desmonta o módulo.

Ao gravar

A máscara é obrigatória e vive **fora** do conteúdo. Duas travas rejeitam máscara que revele o valor: não pode ser igual ao valor, e precisa ter marca de ocultação (**•** ou *****).

PARTE 5 · O QUE CONSTRUIR AGORA, EM ORDEM

#	Tarefa	Depende de
1	Rodar os 8 arquivos SQL no Supabase, na ordem	—
2	Ativar o hook no painel e verificar	1
3	Rodar os testes de aceite e guardar a saída	2
4	Motor gravando em farus_itens em vez de arquivo	3
5	Montador lendo do FARUS	4
6	Agendamento em nuvem às 06:45	5
7	Front-end ligado ao banco	2

Os itens 1 a 3 somam menos de uma hora e transformam "banco instalado" em "sistema funcionando".

PARTE 6 · REGRAS QUE NÃO SE NEGOCIAM

Regra	Por quê
service_role nunca vai ao navegador	Ignora todas as travas. Vazou, todo o banco de todos os clientes está aberto
Nenhuma consulta filtra tenant_id no código	Se o filtro depende do código, um esquecimento vira vazamento. O banco nega mesmo que o programa peça errado
Zero innerHTML com dado	Porta de injeção de script. Use textContent e createElement — ver web/js/seguro.js
Validação no navegador é conveniência, não segurança	Pode ser contornada. A validação que vale é a do banco
Credencial nominal, nunca compartilhada	Sem nome, a auditoria não responde <i>quem fez</i>
Trabalhe em ambiente de teste com dados fictícios	Dado real de gabinete exige autorização expressa

PARTE 7 · ARMADILHAS QUE JÁ CUSTARAM TEMPO

Teste de RLS rodado como superusuário mente. Superusuário ignora RLS. Teste de isolamento exige transação explícita e `set local role authenticated`. Sem isso, tudo "passa" e nada está provado.

Inserção que falha em silêncio parece defeito do código testado. Confira os nomes das colunas: `tenants` usa `nome_autoridade` e `municipio_sede`, não `nome` e `municipio`.

IMMUTABLE sobre função STABLE corrompe índice silenciosamente. Aconteceu com `farus_hash` chamando `unaccent`. Corrigido com `translate`. Se criar função para índice, confirme a volatilidade real de tudo que ela chama.

Consulta vazia quando deveria trazer dados quase sempre é token sem tenant — não a consulta. Verifique o hook antes de mexer no código.

PARTE 8 · O QUE AINDA NÃO EXISTE

Declarado para não se gastar tempo procurando:

Conector real de Diário Oficial · endpoint do PNCP confirmado em produção · Aegis A1 (cifragem em envelope) e A2 (quórum) · Sigma One · radar autônomo (F3) · módulo Demandas (desligado até parecer jurídico) · cabeçalhos de segurança web · limite de tentativas de login · encadeamento por hash na trilha.

*Caderno do Programador · 02/09/2026 · acompanha o Manual Supremo v10.9.1 Évora Oversight ·
Confidencial — Eng. de Sistemas Luiz Gonzaga Filho Protegido pelas Leis nº 9.610/98 e nº 9.609/98*